



Computer Architecture and Operating System

Cache, Intro to OS

Lecturer: Tianming Bai



Review

- Types of Memory
- Memory Hierarchy and Locality
- Cache Memory: Fundamentals, Design & Performance



Content

- Complete remaining cache memory topics
- Introduction to Virtual Memory
- Introduction to Operating System



Recall – Cache Mapping



Cache Write Policies

When the CPU modifies data in the cache, how do we maintain data consistency between the cache and main memory?

Core Definition

Dirty Block: A cache block that has been modified by the CPU, but not yet updated to the main memory.

Two Industry-Standard Policies

- Write-Through Policy
- Write-Back (Copy-Back) Policy



Write-Through Policy

Every CPU write to the cache is **immediately and synchronously written to main memory**, at the same time the cache block is updated.

Step-by-Step Workflow

1. CPU generates a write request
2. Cache hit: update the cache block, *immediately write the identical data to main memory*
3. Cache miss: default to Non-Write Allocate (write directly to main memory, do not load the block into cache)



Write-Back (Copy-Back) Policy

CPU writes are only applied to the cache block initially. Main memory is updated *only when the dirty cache block is about to be evicted*.

Hardware Support:

1-bit **Dirty Bit** stored with every cache block:

- Set to 1 when the CPU modifies the cached data
- Checked during eviction: if 1, write the full block back to main memory before overwriting

Step-by-Step Workflow

- CPU generates a write request
- Cache hit: update the cache block, set the Dirty Bit to 1 (no main memory access needed)
- Cache miss: default to Write Allocate (load the missing block into cache, update it, then set the Dirty Bit to 1)



Write Policy Comparison

Feature	Write-Through	Write-Back
Main Memory Update Timing	Immediate, on every CPU write	Delayed, only when the block is evicted
Dirty Bit Required	No	Yes
Main Memory Traffic	High	Low
Hardware Complexity	Low	High
Write Performance	Limited by main memory speed	Full cache-speed writes
Data Consistency Guarantee	Guaranteed	Only guaranteed on block eviction



Effective Access Time

Effective Access Time (EAT) is the weighted average time to access memory, accounting for both cache hits and misses. It is the single most important metric for real-world memory system performance.

Standard EAT Formula

$$EAT = (Hit\ Rate \times Hit\ Time) + (Miss\ Rate \times Miss\ Penalty)$$

where

Hit Time: Time to access data in the cache (1-3ns for L1 cache in modern CPUs)

Miss Penalty: Total time to process a cache miss, including fetching the block from main memory (50-100ns for modern DRAM)

$$Hit\ Rate + Miss\ Rate = 1$$



EAT Calculation Example

L1 Cache *Hit Time* = $2ns$, *Hit Rate* = 95%

Main Memory *Access Time (Miss Penalty)* = $80ns$

1. Compute EAT
2. If we improve the hit rate to 99%, compute EAT



EAT Calculation Example

L1 Cache *Hit Time* = $2ns$, *Hit Rate* = 95%

Main Memory *Access Time (Miss Penalty)* = $80ns$

Calculation

1. $EAT = (0.95 \times 2ns) + (0.05 \times 80ns) = 1.9ns + 4ns = 5.9ns$
2. $EAT = (0.99 \times 2ns) + (0.01 \times 80ns) = 1.98ns + 0.8ns = 2.78ns$

A **4% improvement in cache hit rate** cuts the average memory access time in half! Small improvements in hit rate lead to massive gains in overall system performance.



Wrap-Up

Cache Memory

1. Memory hierarchy and the principle of locality
2. Cache mapping schemes (Direct-Mapped, Fully Associative, Set Associative)
3. Cache write policies and consistency
4. Performance measurement (EAT)
5. Cache replacement policies



Wrap-Up

Cache Memory

1. Memory hierarchy and the principle of locality
2. Cache mapping schemes (Direct-Mapped, Fully Associative, Set Associative)
3. Cache write policies and consistency
4. Performance measurement (EAT)
5. Cache replacement policies

We have a fast cache system, but **what if our program is larger than the physical RAM?** What if we want to run multiple programs at the same time, without them overwriting each other's memory?



Virtual Memory

Core Limitations of Physical-Only Memory

1. **Program size limit:** A program's size is strictly capped by the amount of physical RAM in the system
2. **No memory isolation:** Multiple running programs can overwrite each other's data, causing crashes or security risks
3. **No memory protection:** A single buggy program can crash the entire system
4. **No portability:** Programs must be rewritten to run on systems with different amounts of RAM

What Virtual Memory Solves

It creates the **illusion of a large, continuous, private memory space for every running program**, using a combination of physical RAM and disk storage.



Virtual Memory

Virtual Memory abstracts physical RAM into a logical, virtual address space for each program. The system handles translating these virtual addresses to real physical addresses.

Key Definitions

- **Virtual Address:** The logical address used by the running program (generated by the CPU)
- **Physical Address:** The actual, hardware address in the physical RAM chip
- **Address Translation:** The process of converting a virtual address to a physical address
- **Page:** Fixed-size chunk of virtual memory (typically 4KB in modern systems)
- **Frame:** Fixed-size chunk of physical memory (same size as a page)



Cache vs Virtual Memory

Component	Implementation	Who Manages It?	Core Goal
Cache Memory	100% hardware implementation	CPU cache controller (hardware only)	Minimize average memory access time
Virtual Memory	Hardware + software collaboration	Operating System (OS) + CPU MMU (Memory Management Unit)	Extend memory capacity, provide isolation/protection, and simplify programming

Which means:

- Cache operations are completely invisible to the OS and programs, the hardware handles everything automatically.
- Virtual Memory cannot work without an Operating System. The OS is responsible for:
 1. Managing page tables (the map of virtual → physical addresses)
 2. Handling page faults (when a page is on disk instead of RAM)
 3. Allocating and isolating memory between running programs
 4. Managing disk swap space



From Hardware to Software

- For the first 4 weeks, we focused on **hardware**: CPU architecture, ISA, pipelining, cache memory, and physical memory systems.
- Starting today, we shift our focus to the **software layer that manages all this hardware**: the Operating System.



From Hardware to Software

The Von Neumann Model Basics

A running program does one simple thing, billions of times per second:

- **Fetch:** The processor fetches an instruction from memory
- **Decode:** It figures out what the instruction is
- **Execute:** It performs the operation (math, memory access, condition check, etc.)
- Repeat, until the program finishes.

While a program runs, the OS is working constantly in the background. It makes it easy to run multiple programs at once, share memory safely, interact with devices, and more – all while the program runs.



The Operating System

Definition:

The Operating System is a body of software responsible for:

1. Making the system easy to use for running programs
2. Virtualizing physical hardware resources into a more powerful, easy-to-use virtual form
3. Managing shared system resources safely and efficiently between programs
4. Handling persistent storage, device I/O, and concurrency



Three Core Themes

This course will follow the OSTEP textbook structure, organized into three "easy pieces":

- **Virtualization:** How the OS virtualizes the CPU and memory
- **Concurrency:** How the OS handles parallel execution safely and correctly
- **Persistence:** How the OS stores data reliably on persistent storage devices



Virtualization of the CPU

Example

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/time.h>
4  #include <assert.h>
5  #include "common.h"
6
7  int
8  main(int argc, char *argv[])
9  {
10     if (argc != 2) {
11         fprintf(stderr, "usage: cpu <string>\n");
12         exit(1);
13     }
14     char *str = argv[1];
15     while (1) {
16         Spin(1);
17         printf("%s\n", str);
18     }
19     return 0;
20 }
```

Figure 2.1: Simple Example: Code That Loops And Prints (`cpu.c`)



Virtualization of the CPU

Example

```
prompt> gcc -o cpu cpu.c -Wall
prompt> ./cpu "A"
A
A
A
A
^C
prompt>
```



Virtualization of the CPU

Example

```
prompt> ./cpu A & ./cpu B & ./cpu C & ./cpu D &  
[1] 7353  
[2] 7354  
[3] 7355  
[4] 7356  
A  
B  
D  
C  
A  
B  
D  
C  
A  
...
```



Virtualization of the CPU

Example

If we run 4 instances of the same looping print program on a single CPU:

- The OS rapidly switches the CPU between each program
- To the user, it looks like all 4 programs are running simultaneously
- This is called **time-sharing** the CPU



Virtualization of the CPU

The OS turns a single physical CPU (or small set of CPUs) into the illusion of an infinite number of virtual CPUs. This lets many programs seemingly run at the same time, even on a single-core processor.



Virtualization of Memory

Example

```
1  #include <unistd.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include "common.h"
5
6  int
7  main(int argc, char *argv[])
8  {
9      int *p = malloc(sizeof(int));           // a1
10     assert(p != NULL);
11     printf("(%d) address pointed to by p: %p\n",
12           getpid(), p);                      // a2
13     *p = 0;                                  // a3
14     while (1) {
15         Spin(1);
16         *p = *p + 1;
17         printf("(%d) p: %d\n", getpid(), *p); // a4
18     }
19     return 0;
20 }
```

Figure 2.3: A Program That Accesses Memory (`mem.c`)



Virtualization of Memory

Example

```
prompt> ./mem
(2134) address pointed to by p: 0x200000
(2134) p: 1
(2134) p: 2
(2134) p: 3
(2134) p: 4
(2134) p: 5
^C
```



Virtualization of Memory

Example

```
prompt> ./mem & ./mem &  
[1] 24113  
[2] 24114  
(24113) address pointed to by p: 0x200000  
(24114) address pointed to by p: 0x200000  
(24113) p: 1  
(24114) p: 1  
(24114) p: 2  
(24113) p: 2  
(24113) p: 3  
(24114) p: 3  
(24113) p: 4  
(24114) p: 4  
...
```



Virtualization of Memory

The OS gives every running program its own private, continuous virtual address space. To the program, it looks like it has full, exclusive access to the system's memory – even though physical RAM is shared between dozens of programs.



Virtualization of Memory

Example

If we run 2 instances of the same memory program:

- Both programs see their allocated memory at the *exact same virtual address* (e.g., 0x200000)
- Each program updates the value at this address independently, with no impact on the other
- The OS maps each program's virtual address to a different location in physical RAM, invisibly

Key OS Responsibilities for Memory Virtualization

1. Manage page tables for every running program
2. Handle page faults and disk swap space
3. Enforce memory protection and isolation between programs
4. Allocate and free memory for programs on request



Concurrency

Example

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "common.h"
4  #include "common_threads.h"
5
6  volatile int counter = 0;
7  int loops;
8
9  void *worker(void *arg) {
10     int i;
11     for (i = 0; i < loops; i++) {
12         counter++;
13     }
14     return NULL;
15 }
16
17 int main(int argc, char *argv[]) {
18     if (argc != 2) {
19         fprintf(stderr, "usage: threads <value>\n");
20         exit(1);
21     }
22     loops = atoi(argv[1]);
23     pthread_t p1, p2;
24     printf("Initial value : %d\n", counter);
25
26     Pthread_create(&p1, NULL, worker, NULL);
27     Pthread_create(&p2, NULL, worker, NULL);
28     Pthread_join(p1, NULL);
29     Pthread_join(p2, NULL);
30     printf("Final value   : %d\n", counter);
31     return 0;
32 }
```

Figure 2.5: A Multi-threaded Program (**threads.c**)



Concurrency

Example

```
prompt> gcc -o threads threads.c -Wall -pthread
prompt> ./threads 1000
Initial value : 0
Final value   : 2000
```



Concurrency

Example

```
prompt> ./threads 100000  
Initial value : 0  
Final value   : 143012    // huh??  
prompt> ./threads 100000  
Initial value : 0  
Final value   : 137298    // what the??
```



Concurrency

Concurrency refers to the set of problems that arise when working on many things at once (in parallel) in the same program or system.

Where Concurrency Happens

1. Inside the OS itself: The OS is juggling dozens of programs at once, switching between them constantly
2. Inside modern multi-threaded programs: Multiple threads of execution run within the same memory space, at the same time



Concurrency

If two threads increment the same shared counter 100,000 times each, you do not always get 200,000 as the final result.

- Why? The increment operation takes 3 separate instructions (load, increment, store), which do not execute all at once.
- The OS must provide tools to handle this correctly: synchronization primitives like locks, semaphores, and condition variables.



Persistence

Example

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <assert.h>
4  #include <fcntl.h>
5  #include <sys/types.h>
6
7  int main(int argc, char *argv[]) {
8      int fd = open("/tmp/file",
9                    O_WRONLY|O_CREAT|O_TRUNC,
10                   S_IRWXU);
11      assert(fd > -1);
12      int rc = write(fd, "hello world\n", 12);
13      assert(rc == 12);
14      close(fd);
15      return 0;
16 }
```



Persistence

Data in RAM is volatile – it is lost when power is cut. We need a way to store data permanently, even when the system is turned off.

The OS Solution: The File System

- The OS manages persistent storage devices (HDDs, SSDs) via the file system. It is responsible for storing user files reliably and efficiently on disk.



Persistence

Key OS Responsibilities for Persistence

1. Translate file operations (open, read, write, close) into low-level I/O requests to the storage device
2. Ensure data reliability, even if the system crashes mid-write (via journaling, copy-on-write, etc.)
3. Organize data into a hierarchical file/folder structure
4. Enforce file permissions and sharing between users and programs



Core Takeaways

- We completed all cache memory topics: write policies, EAT calculation
- **Virtual Memory** solves the limitations of physical-only memory, and requires OS + hardware collaboration to work
- The OS has two core roles: a virtual machine that abstracts hardware, and a resource manager that shares system resources safely
- The four core pillars of modern OS are: CPU Virtualization, Memory Virtualization, Concurrency, and Persistence
- System calls are the standard interface for applications to request services from the OS



Next Lecture

What is Process?

